

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

кафедра програмних засобів

ЗВІТ

з лабораторної роботи № 2

з дисципліни «Технології розподілених систем та паралельних
обчислень» на тему:

«ПОБУДОВА ПАРАЛЕЛЬНИХ АЛГОРИТМІВ»

Виконав:

ст. гр. КНТ-113сп

Іван ЩЕДРОВСЬКИЙ

Прийняв:

Старший викладач

Олександр КАЧАН

1 Мета роботи

Одержати навички програмування багатопоточних додатків в WIN32 на прикладі алгоритмів модульного піднесення до степені.

2 Завдання до лабораторної роботи

2.1 Реалізувати бінарний метод модульного піднесення до степені.

2.2 Реалізувати двухпоточний варіант алгоритму Монтгомері.

2.3 Реалізувати метод гребеня модульного піднесення до степені. Для методу гребеня число потоків повинне вводитися із клавіатури під час виконання програми.

2.4 Порівняти час виконання операції модульного піднесення до степені трьома реалізованими методами.

3 Виконання лабораторної роботи

Для виконання лабораторної роботи було використано мову C++.

Для виконання першого завдання було використано метод “Square and Multiply”, тобто модульного піднесення до степені.

Суть цього алгоритму полягає в двох головних ідеях або основах.

Перша основа це зменшення кількості операцій піднесення до степені за рахунок додавання цих степенів. Наприклад, якщо нам потрібно представити число 2^8 , то замість того, щоб множити 2 само на себе 8 разів ми можемо зробити 2^{2^2} . Таким чином ми зменшили кількість операцій з 7 множень всього до 3-х.

Друга основа полягає в тому, що всі числа на комп'ютері зберігаються в бінарному форматі. Оскільки всі числа зберігаються в бінарному форматі, то ми можемо представити будь-яке число через суму двійок та одиниць.

Головна ідея алгоритму створити ступінь використовуючи мінімальну кількість операцій. Алгоритм починає свою роботу зліва на право по бінарному числу ступені. Результат перед початком алгоритму дорівнює 1. При кожній операції виконується підведення результату степені 2(або його множення саме на себе). В випадку коли цифра поточного індексу дорівнює 1 додатково виконується множення на початкове число a .

Таким чином алгоритм один за одним кроком вибудовує ступінь, яка нам потрібна. Також варто зауважити одне цікаве число 65537 яке використовується в криптографії, а саме часто в RSA, оскільки RSA побудований на основі ступенів за модулями. Число 65537 цікаве тим, що його бінарне представлення виглядає як 100000000000000001, і оскільки в ньому так багато нулів цей алгоритм може за мінімальну кількість кроків визначити його значення.

Другим алгоритмом є “Montgomery Ladder”. Оригінально цей алгоритм був створений для знаходження точок на еліптичних кривих. Також важливою характеристикою цього алгоритму є захист від атак по енергоспоживанню, оскільки раніше в пластикових картках можна було за енергоспоживанням зрозуміти секретний ключ.

Оскільки алгоритм повинен працювати на декількох потоках було створено дві його версії – перша без тредів, а друга з тредями, які створюються всередині циклу для розрахунку наступного значення y_1 та y_2 . В поточному в поточній реалізації алгоритмі використання паралельності буде навпаки сповільнювати його, оскільки створення нових потоків та виконання операцій на них буде довше, а ніж виконати операції напрямку.

І останній алгоритм це алгоритм гребеня. Цей метод розбиває бітовий рядок ступені на паралельні блоки. Це дозволяє замінити послідовну обробку кожного біта на вибірку попередньо обчислених значень із таблиці, що скорочує кількість циклічних операцій квадратування та множення.

На рисунку 1 показано порівняння швидкостей алгоритмів при їх запуску на 100 ітерацій. На рисунку 2 показано результати запуску програми на 100 ітерацій.

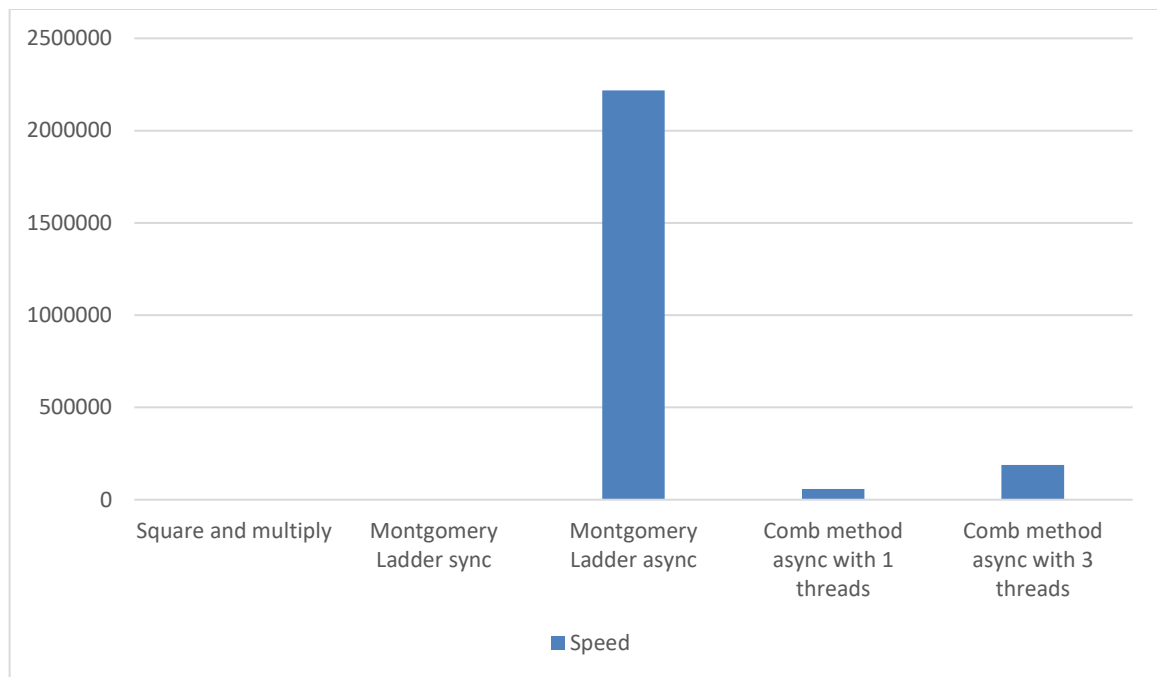


Рисунок 1 – Графік порівняння швидкостей

```
y = 69234^65537 mod 6784  
Expected answer = 4480
```

```
Square and multiply: 17 microseconds | Result: 4480  
Montgomery Ladder sync: 39 microseconds | Result: 4480  
Montgomery Ladder async: 2217673 microseconds | Result: 4480  
Comb method async with 1 threads: 57754 microseconds | Result: 4480  
Comb method async with 3 threads: 188537 microseconds | Result: 4480
```

Рисунок 2 – Виконання програми

Як можна побачити, то алгоритм «Montgomery Ladder» з використанням паралельності займає набагато більше часу, а ніж без неї.

Далі було вимкнено цей алгоритм, а кількість запусків збільшено до 1000. Графік результатів показаний на рисунку 3, а виконання показано на рисунку 4.

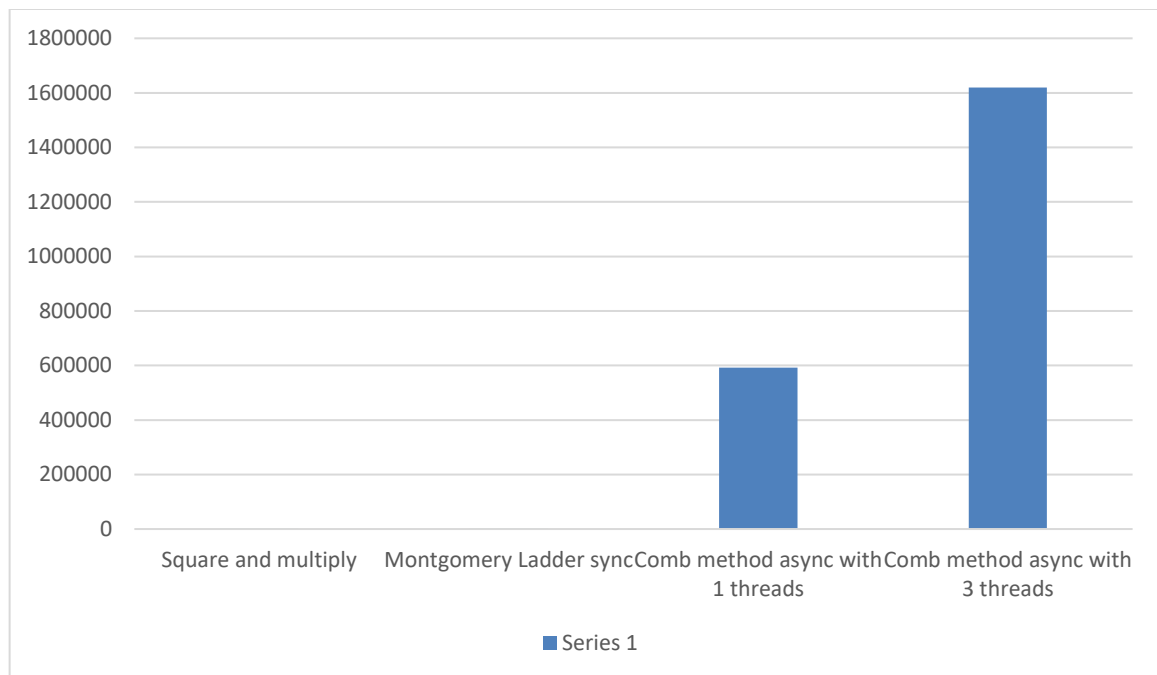


Рисунок 1 – Графік порівняння швидкостей при 1000 операцій

```
y = 69234^65537 mod 6784
Expected answer = 4480
```

```
Square and multiply: 217 microseconds | Result: 4480
Montgomery Ladder sync: 418 microseconds | Result: 4480
Comb method async with 1 threads: 591515 microseconds | Result: 4480
Comb method async with 3 threads: 1619410 microseconds | Result: 4480
```

Рисунок 2 – Виконання програми при 1000 операцій

Як можна побачити найбільш ефективним алгоритмом є Square and Multiply. В обчисленнях швидкості в поточній лабораторній роботі є проблеми, оскільки робота відбувалась на відносно не великих значеннях. Цілком ймовірно, що якщо значення будуть великі, наприклад числа в дві або чотири тисячі біт, то результати будуть сильно відрізнятись.

4 Тест розробленої програми

```
#include <iostream>
#include <windows.h>
#include <process.h>
#include <chrono>
```

```

int getBitLength(int x) {
    int count = 0;

    while (x > 0) {
        x = x >> 1;
        count++;
    }

    return count;
}

// y = a^n mod m

int squareAndMultiply(int a, int n, int m) {
    int y = 1;
    int bits = getBitLength(n);

    for (int i = bits - 1; i >= 0; i--) {
        y = (y * y) % m;

        if (n >> i & 1) {
            y = (y * a) % m;
        }
    }

    return y;
}

int montgomeryLadderSync(int a, int n, int m) {
    int y1 = 1;
    int y2 = (a) % m;

    int N = getBitLength(n);

    for (int i = N - 1; i >= 0; i--) {
        if (n >> i & 1) {
            y1 = (1LL * y1 * y2) % m;
            y2 = (1LL * y2 * y2) % m;
        } else {
            y2 = (1LL * y1 * y2) % m;
            y1 = (1LL * y1 * y1) % m;
        }
    }

    return y1;
}

int montgomeryLadderAsync(int a, int n, int m) {
    int y1 = 1;
    int y2 = (a) % m;

    int N = getBitLength(n);

    for (int i = N-1; i >= 0; i--) {
        int bit = n >> i & 1;

        int data1[5] = { y1, y2, bit, m, 0 };
        int data2[5] = { y1, y2, bit, m, 0 };

        auto proc1 = [](void* p) {
            int* d = (int*)p;

            if (d[2] == 1) d[4] = (1LL * d[0] * d[1]) % d[3];
            else d[4] = (1LL * d[0] * d[0]) % d[3];
        };
    }
}

```

```

};

auto proc2 = [](void* p) {
    int* d = (int*)p;
    if (d[2] == 1) d[4] = (1LL * d[1] * d[1]) % d[3];
    else          d[4] = (1LL * d[0] * d[1]) % d[3];
};

HANDLE h[2];
h[0] = (HANDLE)_beginthread(proc1, 0, data1);
h[1] = (HANDLE)_beginthread(proc2, 0, data2);

WaitForMultipleObjects(2, h, TRUE, INFINITE);

y1 = data1[4];
y2 = data2[4];
}

return y1;
}

int power(int base, int exp, int mod) {
    int res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1) res = (1LL * res * base) % mod;
        base = (1LL * base * base) % mod;
        exp /= 2;
    }
    return res;
}

struct TableData {
    int a;
    int exp;
    int m;
    int result;
};

void __cdecl calcBaseValue(void* p) {
    TableData* d = (TableData*)p;
    d->result = power(d->a, d->exp, d->m);
}

int combMethodAsync(int a, int n, int m, int numThreads) {
    int w = numThreads;
    if (w > 16) w = 16;

    int bits = getBitLength(n);
    int h = (bits + w - 1) / w;
    if (h == 0) h = 1;

    int tableSize = 1 << w;
    int* table = new int[tableSize];
    table[0] = 1;

    TableData* tData = new TableData[w];
    HANDLE* threads = new HANDLE[w];

    for (int i = 0; i < w; i++) {
        tData[i] = { a, (1 << (i * h)), m, 0 };
        threads[i] = (HANDLE)_beginthread(calcBaseValue, 0, &tData[i]);
    }

    WaitForMultipleObjects(w, threads, TRUE, INFINITE);
}

```

```

    for (int i = 0; i < w; i++) {
        int val = tData[i].result;
        int step = 1 << i;
        for (int j = 0; j < step; j++) {
            int targetIndex = step + j;

            if (targetIndex < tableSize) {
                table[targetIndex] = (1LL * table[j] * val) % m;
            }
        }
    }

    int Y = 1;
    for (int i = h - 1; i >= 0; i--) {
        Y = (1LL * Y * Y) % m;

        int index = 0;
        for (int j = 0; j < w; j++) {
            if ((n >> (j * h + i)) & 1) {
                index |= (1 << j);
            }
        }

        if (index > 0) {
            Y = (1LL * Y * table[index]) % m;
        }
    }

    delete[] table;
    delete[] tData;
    delete[] threads;

    return Y;
}

void measure(const char* title, int (*func)(int, int, int), int a, int n, int m) {
    auto start = std::chrono::high_resolution_clock::now();

    int result = 0;
    for (int i = 0; i < 1000; i++) {
        result = func(a, n, m);
    }

    auto end = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start);

    std::cout << title << ": " << duration.count() << " microseconds | Result: " <<
result << std::endl;
}

void measureComp(const char* title, int (*func)(int, int, int, int), int a, int n, int m,
int numThreads) {
    auto start = std::chrono::high_resolution_clock::now();

    int result = 0;
    for (int i = 0; i < 1000; i++) {
        result = func(a, n, m, numThreads);
    }

    auto end = std::chrono::high_resolution_clock::now();
    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end - start);

    std::cout << title << ": " << duration.count() << " microseconds | Result: " <<
result << std::endl;
}

```



```

}

int main() {
    const int A = 69234;
    const int N = 65537;
    const int M = 6784;

    std::cout << "y = " << A << "^" << N << " mod " << M << "\n";
    std::cout << "Expected answer = 4480\n";

    std::cout << std::endl;

    measure("Square and multiply", squareAndMultiply, A, N, M);
    measure("Montgomery Ladder sync", montgomeryLadderSync, A, N, M);
    measure("Montgomery Ladder async", montgomeryLadderAsync, A, N, M);
    measureComp("Comb method async with 1 threads", combMethodAsync, A, N, M, 1);
    measureComp("Comb method async with 3 threads", combMethodAsync, A, N, M, 3);

    return 0;
}

```

5 Висновки

Я ознайомився із застосуванням різних методів синхронізації при створенні багатопоточних застосунків